



WIESŁAW PIETRUSZKIEWICZ

ARCHITECTURE OF WEB SYSTEMS

Client

- **Web Browser** – The interface through which users interact with web applications. It renders HTML, CSS, and JavaScript to display content.
- **Mobile Applications** – Native or hybrid apps that communicate with web services to provide functionality.

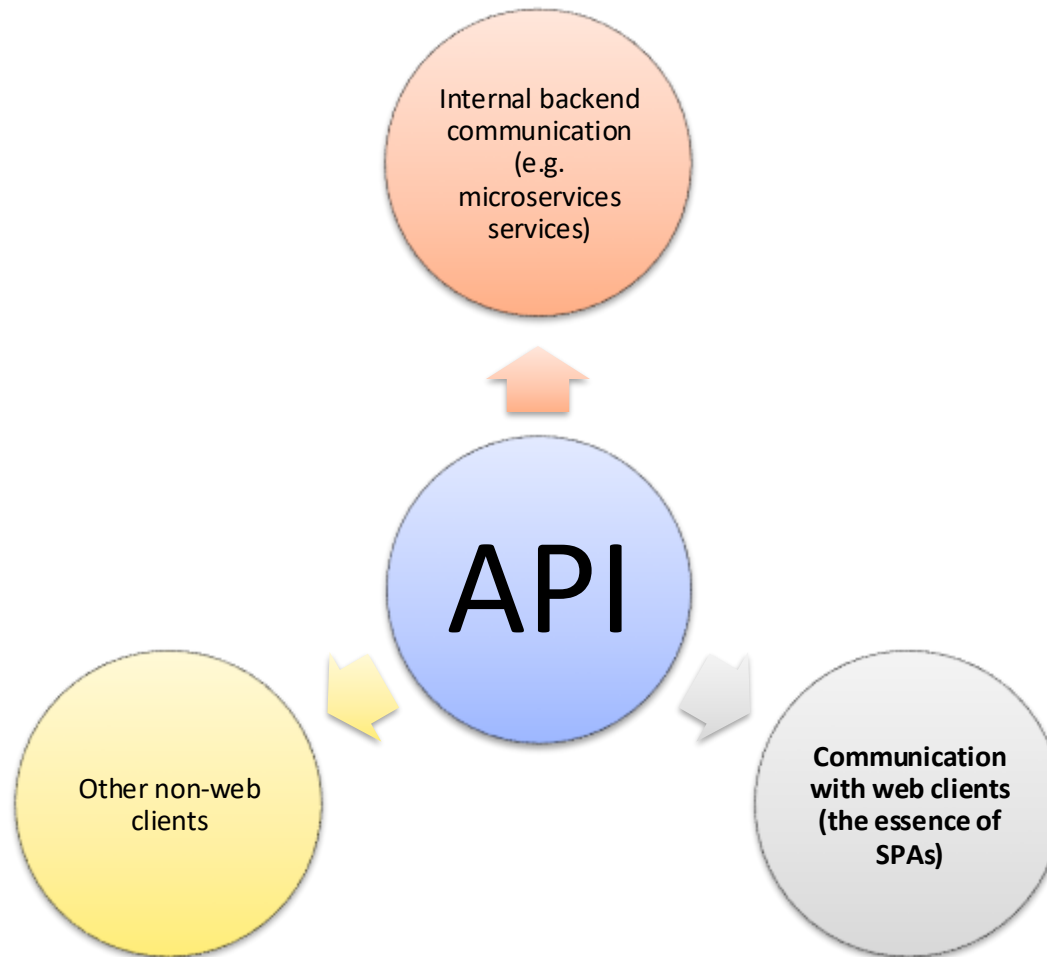
Web server / Main part of back-end

- **HTTP Server** – Handles incoming requests from clients and serves static content (HTML, CSS, images) or forwards requests to application servers (e.g. Apache, Nginx or Microsoft IIS).
- **Application Server** – Processes dynamic content and business logic, often running server-side scripts or applications (Tomcat, Netty, Uvicorn or Node.js).

API (Application Programming Interface)

- **RESTful API (Representational State Transfer)**
 - Enable communication between the client and server (usually), allowing clients to request and manipulate data.
- **GraphQL** – An alternative to REST that allows clients to request specific data structures.

Common usage of API



Restful API

- A web service that adheres to the principles and constraints of REST architecture.
- REST is an architectural style for designing networked applications.
- Relies on a stateless client-server communication model.
- Each request from a client to a server must contain all the information needed to understand and process the request.

Restful API – Methods

- RESTful APIs commonly use the following HTTP methods to perform operations on resources:
 - GET – Retrieve data from the server
 - POST – Create a new resource on the server.
 - PUT – Update an existing resource or create a new resource if it does not exist.
 - DELETE – Remove a resource from the server.

HTTP Codes

CODE	STATUS	DESCRIPTION
200	OK	The request has succeeded. The meaning of the success depends on the HTTP method used.
201	Created	The request has been fulfilled, resulting in the creation of a new resource.
204	No Content	The server successfully processed the request, but is not returning any content.
400	Bad Request	The server cannot or will not process the request due to a client error (e.g., malformed request).
401	Unauthorized	Authentication is required and has failed or has not yet been provided.
403	Forbidden	The server understands the request but refuses to authorize it.
404	Not Found	The requested resource could not be found on the server.
405	Method Not Allowed	The request method is not supported for the requested resource.
409	Conflict	The request could not be completed due to a conflict with the current state of the resource.
415	Unsupported Media Type	The request entity has a media type that the server or resource does not support.
500	Internal Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request.
503	Service Unavailable	The server is currently unable to handle the request due to temporary overload or maintenance.

Restful API – Marketstack.com #1

Supported Endpoint

End-of-Day Data

Available on: All plans

You can use the API's `eod` endpoint in order to obtain end-of-day data for one or multiple stock tickers. A single or multiple comma-separated ticker symbols are passed to the API using the `symbols` parameter.

Note: To request end-of-day data for single ticker symbols, you can also use the API's [Tickers Endpoint](#).

Example API Request:

```
https://api.marketstack.com/v1/eod
? access_key = YOUR_ACCESS_KEY
& symbols = AAPL
```

Endpoint Features:

Object	Description
<code>/eod/[date]</code>	Specify a date in <code>YYYY-MM-DD</code> format. You can also specify an exact time in ISO-8601 date format, e.g. <code>2020-05-21T00:00:00+0000</code> . Example: <code>/eod/2020-01-01</code>
<code>/eod/latest</code>	Obtain the latest available end-of-day data for one or multiple stock tickers.

Restful API – Marketstack.com #2

Parameters:

Object	Description
<code>access_key</code>	[Required] Specify your API access key, available in your account dashboard .
<code>symbols</code>	[Required] Specify one or multiple comma-separated stock symbols (tickers) for your request, e.g. <code>AAPL</code> or <code>AAPL,MSFT</code> . Each symbol consumes one API request. Maximum: 100 symbols
<code>exchange</code>	[Optional] Filter your results based on a specific stock exchange by specifying the MIC identification of a stock exchange. Example: <code>XNAS</code>
<code>sort</code>	[Optional] By default, results are sorted by date/time descending. Use this parameter to specify a sorting order. Available values: <code>DESC</code> (Default), <code>ASC</code> .
<code>date_from</code>	[Optional] Filter results based on a specific timeframe by passing a from-date in <code>YYYY-MM-DD</code> format. You can also specify an exact time in ISO-8601 date format, e.g. <code>2020-05-21T00:00:00+0000</code> .
<code>date_to</code>	[Optional] Filter results based on a specific timeframe by passing an end-date in <code>YYYY-MM-DD</code> format. You can also specify an exact time in ISO-8601 date format, e.g. <code>2020-05-21T00:00:00+0000</code> .
<code>limit</code>	[Optional] Specify a pagination limit (number of results per page) for your API request. Default limit value is <code>100</code> , maximum allowed limit value is <code>1000</code> .
<code>offset</code>	[Optional] Specify a pagination offset value for your API request. Example: An offset value of <code>100</code> combined with a limit value of 10 would show results 100-110. Default value is <code>0</code> , starting with the first available result.

Restful API – Marketstack.com #3

```
{
  "pagination": {
    "limit": 100,
    "offset": 0,
    "count": 100,
    "total": 9944
  },
  "data": [
    {
      "open": 129.8,
      "high": 133.04,
      "low": 129.47,
      "close": 132.995,
      "volume": 106686703.0,
      "adj_high": 133.04,
      "adj_low": 129.47,
      "adj_close": 132.995,
      "adj_open": 129.8,
      "adj_volume": 106686703.0,
      "split_factor": 1.0,
      "dividend": 0.0,
      "symbol": "AAPL",
      "exchange": "XNAS",
      "date": "2021-04-09T00:00:00+0000"
    },
    [...]
  ]
}
```

Restful API – Marketstack.com #4

```
const url = "https://api.marketstack.com/v1/eod?access_key={PASTE_YOUR_API_KEY_HERE}&symbols=AAPL";
const options = {
  method: "GET",
};

try {
  const response = await fetch(url, options);
  const result = await response.text();
  console.log(result);
} catch (error) {
  console.error(error);
}
```

```
import requests

url = "https://api.marketstack.com/v1/eod?access_key={PASTE_YOUR_API_KEY_HERE}"

querystring = {"symbols": "AAPL"}

response = requests.get(url, params=querystring)

print(response.json())
```

GraphQL

- GraphQL is a query language for APIs and a runtime for executing those queries with your existing data.
- Developed by Facebook in 2012 and released as an open-source project in 2015.
- Provides a more efficient, powerful, and flexible alternative to the traditional REST API architecture.

GraphQL

- GraphQL uses a single endpoint to handle all requests. This simplifies the API structure and reduces the complexity of managing multiple routes.
- Clients can specify exactly what data they need in a single request. This means that clients can request nested resources and avoid over-fetching or under-fetching data.

GraphQL – graphql.org #1

Operation	Response
<pre>{ hero { name } }</pre>	<pre>{ "data": { "hero": { "name": "R2-D2" } } }</pre>

GraphQL – graphql.org #2

Operation	Response
<pre>{ hero { name friends { name } } }</pre>	<pre>{ "data": { "hero": { "name": "R2-D2", "friends": [{ "name": "Luke Skywalker" }, { "name": "Han Solo" }, { "name": "Leia Organa" }] } } }</pre>

GraphQL – graphql.org #3

Operation	Response
<pre>{ human(id: "1000") { name height } }</pre>	<pre>{ "data": { "human": { "name": "Luke Skywalker", "height": 1.72 } } }</pre>

Synchronous programming languages

- **Tasks are executed sequentially.** Each task must complete before the next one begins. This means that if a task takes time (like a network request or file I/O), the entire program waits for that task to finish before proceeding.
- **When a function is called, the program halts execution until that function returns a result.** This can lead to inefficiencies, especially in applications that require waiting for external resources.
- **Often simpler to understand and reason about because the flow of execution is linear.** Developers can follow the sequence of operations without worrying about the complexities of concurrency.

Asynchronous programming languages

- **Tasks can be executed independently of the main program flow.** When a task is initiated (like a network request), the program can continue executing other tasks without waiting for the initial task to complete.
- **Operations are non-blocking.** When a function is called, it can return immediately, allowing the program to continue executing other code. Once the asynchronous task completes, a callback function, promise, or event can be used to handle the result.
- **Developers must manage the flow of execution and handle callbacks or promises.** This can lead to challenges such as "callback hell," where nested callbacks become difficult to read and maintain.

URL routing

- A mechanism used in web applications to map incoming HTTP requests to specific handlers or controllers based on the URL structure.
- Plays a crucial role in defining how URLs are interpreted and how requests are processed by the server.
- An application server does not provide resources the way HTTP server does.

URL routing – Elements

Routing table

- A collection of rules that define how URLs should be matched to specific actions (methods) or resources (classes) in the application.
- Each rule typically consists of a URL pattern and a corresponding handler (such as a function or controller).

URL Patterns

- Define the structure of the URLs that the application can handle.
- Patterns can include static segments (e.g., /items) and dynamic segments (e.g., /items/{id}), where {id} is a placeholder for a variable value.

HTTP Methods

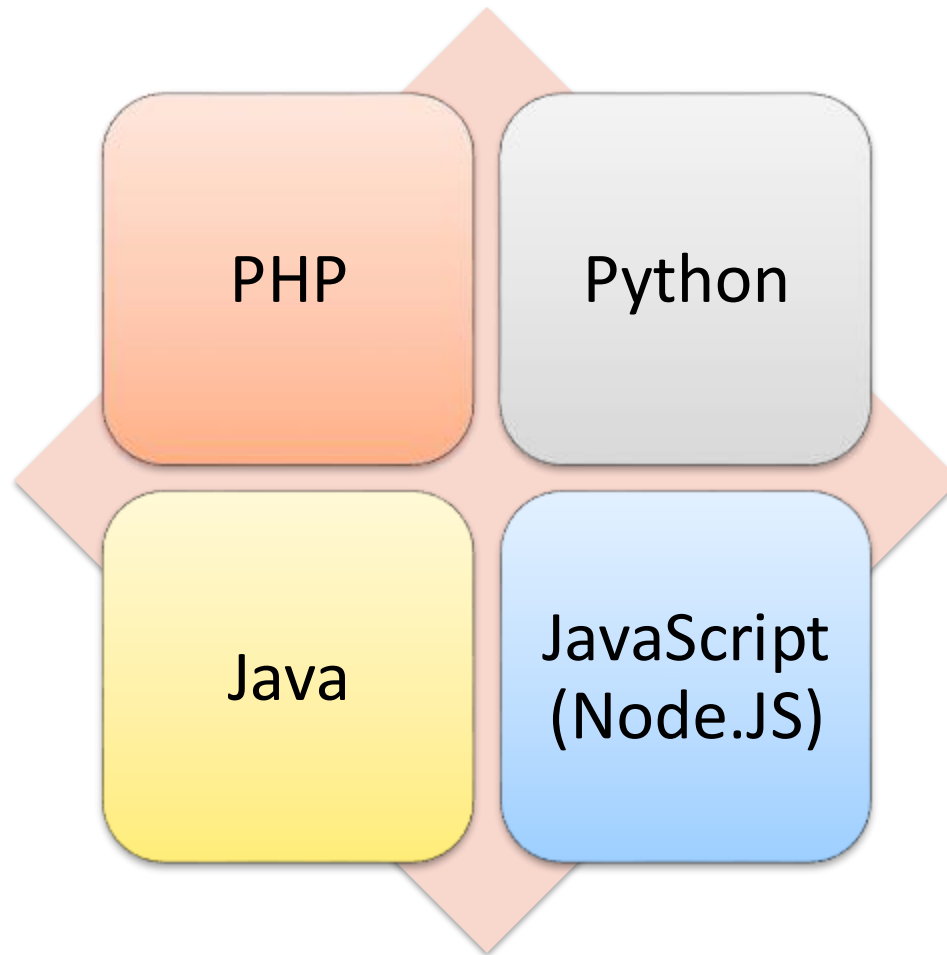
- Routing can also take into account the HTTP method of the request (GET, POST, PUT, DELETE, etc.).
- Different actions can be defined for the same URL based on the method used.

Parameters

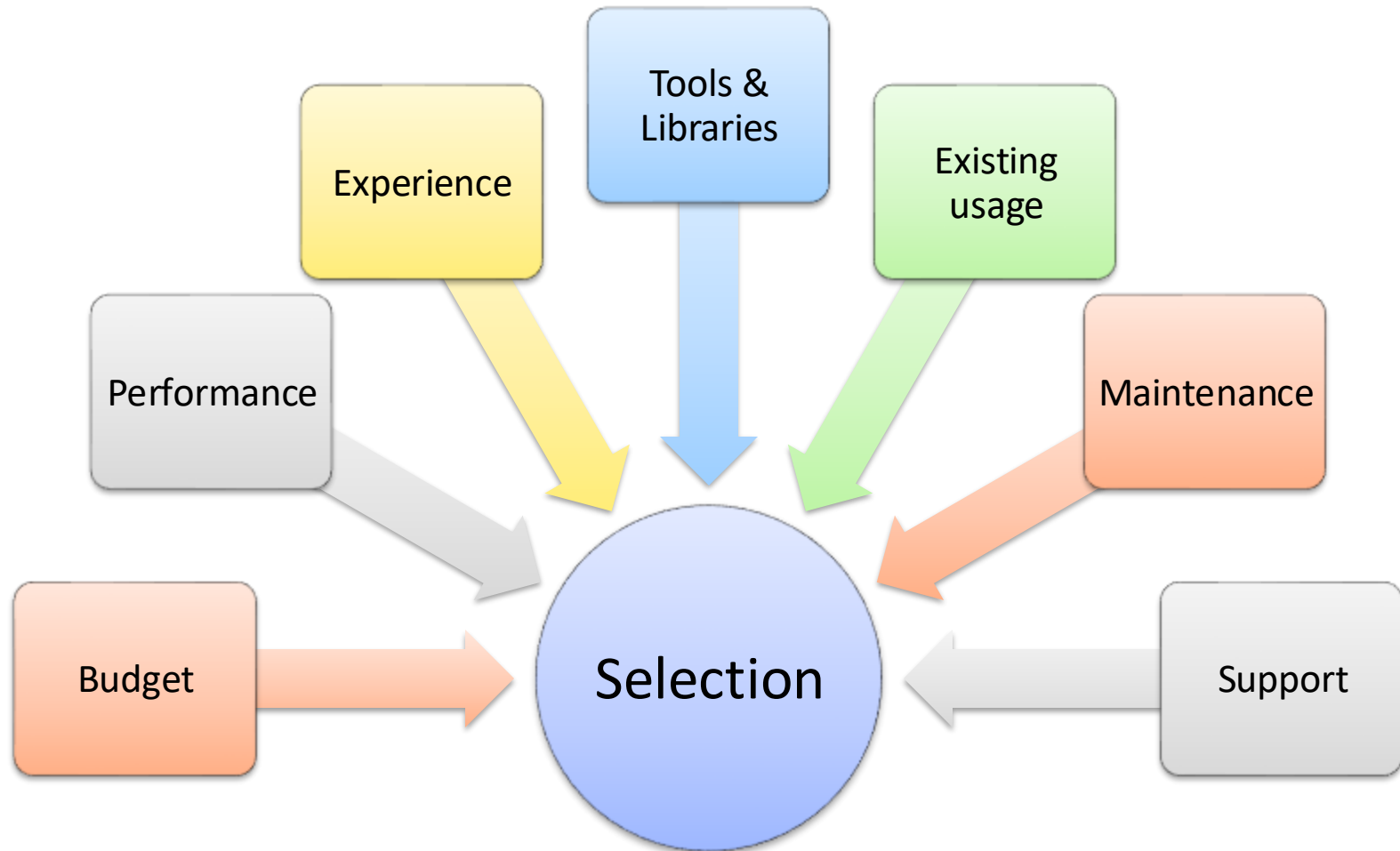
- Dynamic segments in the URL can capture parameters that are passed to the handler.

SERVER-SIDE DYNAMICS OF WEB DOCUMENTS

Popular server-side technologies



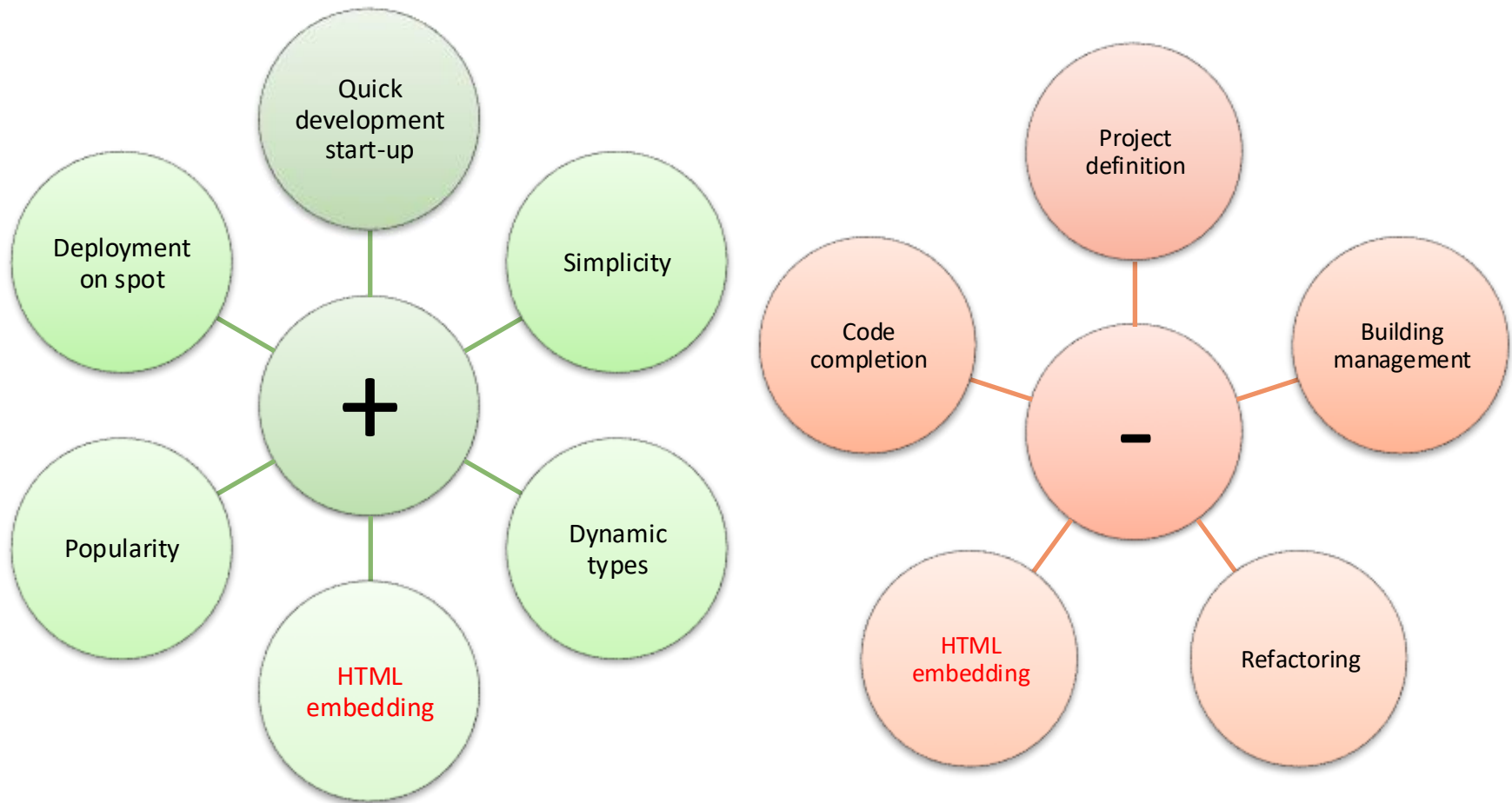
Technology selection



PHP – Basics

- General purpose scripting language oriented towards web development (other usages were unsuccessful)
- Common support by standard hosting packages
- According to W3Techs, ~80% of websites (with known server-side language) uses it
- A foundation layer of many CMSES, e-Commerce or other web-based systems

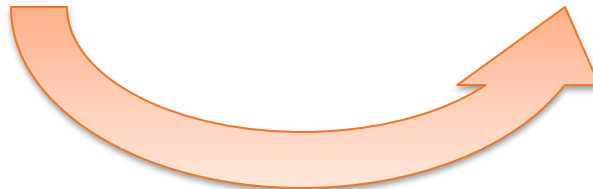
PHP – Pros & Cons



PHP – Hello World

```
<html>
<head>
  <title>PHP Test</title>
</head>
<body>
<?php echo '<p>Hello World</p>'; ?>
</body>
</html>
```

```
<html>
<head>
  <title>PHP Test</title>
</head>
<body>
  <p>Hello World</p>
</body>
</html>
```



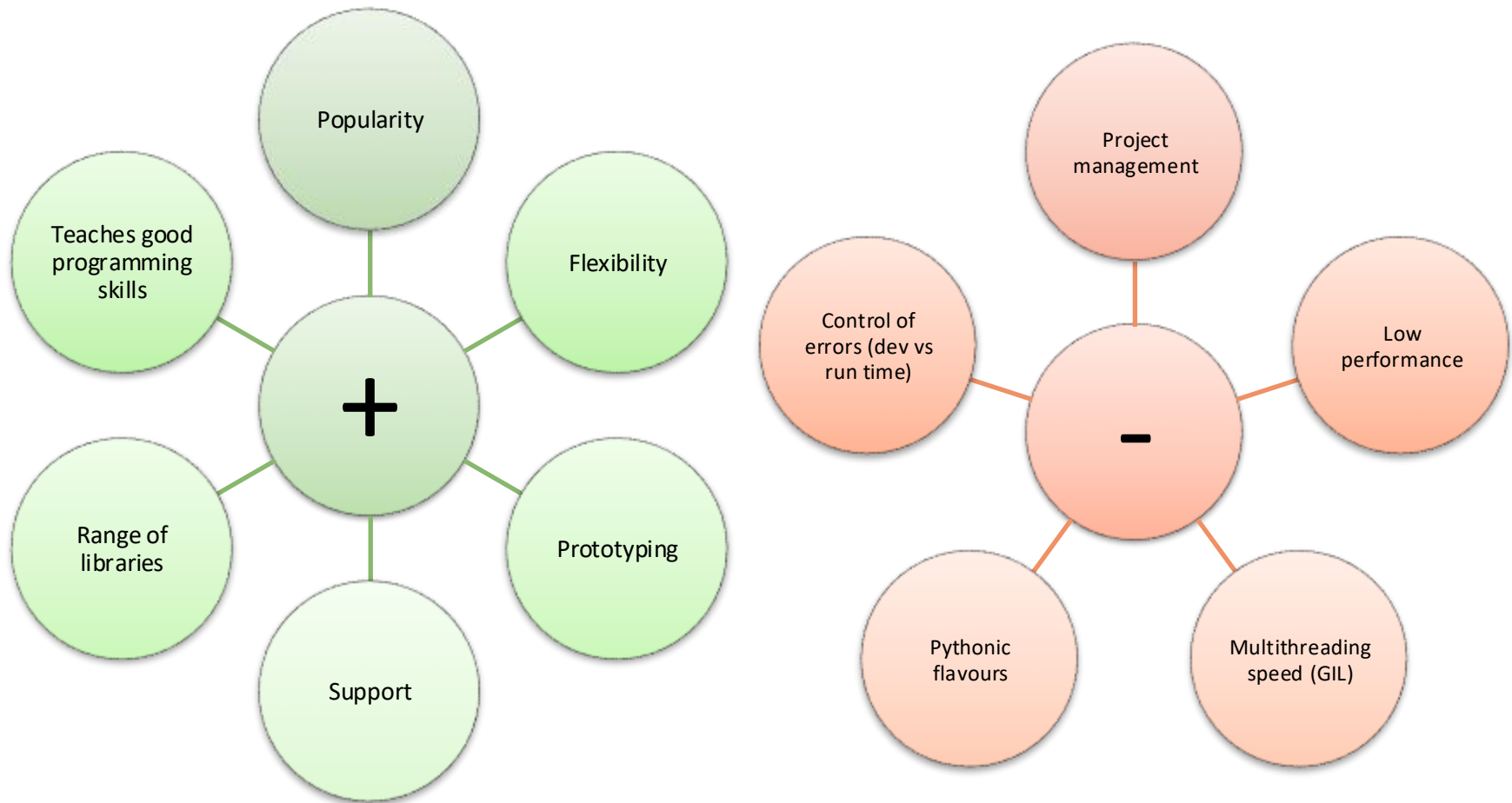
PHP – Complex example

assets	External Libraries: Further fix jQuery deprecations in WordPress core.	12 days ago
inc	Twenty Seventeen: Add support for <code>wa.me</code> links in Social Links menu.	4 months ago
template-parts	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
404.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
archive.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
comments.php	Bundled Themes: Correct <code>@since</code> tags to refer to the theme version i...	2 years ago
footer.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
front-page.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
functions.php	Docs: Synchronize documentation for <code>wp_get_attachment_image_attri...</code>	4 months ago
header.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
index.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
page.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
readme.txt	Bundled Themes: Update the "Tested up to" header to <code>5.9</code> .	23 days ago
rtl.css	Widgets: Add filter to disable RSS widget icon.	last month
screenshot.png	Importing Twenty Seventeen, our new default theme for 2017, set for 4.7.	5 years ago
search.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
searchform.php	Bundled Themes: Correct <code>@since</code> tags to refer to the theme version i...	2 years ago
sidebar.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
single.php	Bundled Themes: Remove the "role" attribute on HTML elements with a d...	last month
style.css	Bundled Themes: Update the "Tested up to" header to <code>5.9</code> .	23 days ago

Python – Basics

- Interpreted general-purpose programming language
- Various applications, including extension to web usage (server-side)
- Focus on a clean, logical code with formatting giving meaning to the parts of code
- *The Swiss Army knife* of programming languages

Python – Pros & Cons



Python – Hello World (Django)

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world. You're at the  
polls index.")
```

Examine a Django example:

<https://docs.djangoproject.com/en/4.0/intro/tutorial01/>

Python – Flask Framework

- Flask is a minimalistic framework i.e. providing functionality to use specialised libraries, not provided modules

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def hello_world():
```

```
    return "<p>Hello, World!</p>"
```

Java – Basics

- A “*default*” option for large-scale systems
- The biggest capabilities
- Complex development environment and deployment
- High requirements for skills and more time consuming dev&ops
- A selection of solid frameworks including:
 - Spring (inc. Boot)
 - Struts
 - Wicket (visual)
 - Play! / Akka (Scala)
 - Grails (Apache Groovy)
 - Hibernate (ORM)

Java – Spring Boot example

```
package com.example.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/")
    public String index() {
        return "Greetings from Spring Boot!";
    }

}
```

Dynamic Documents and Front-end Web Development

```
package com.example.springboot;

import java.util.Arrays;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;

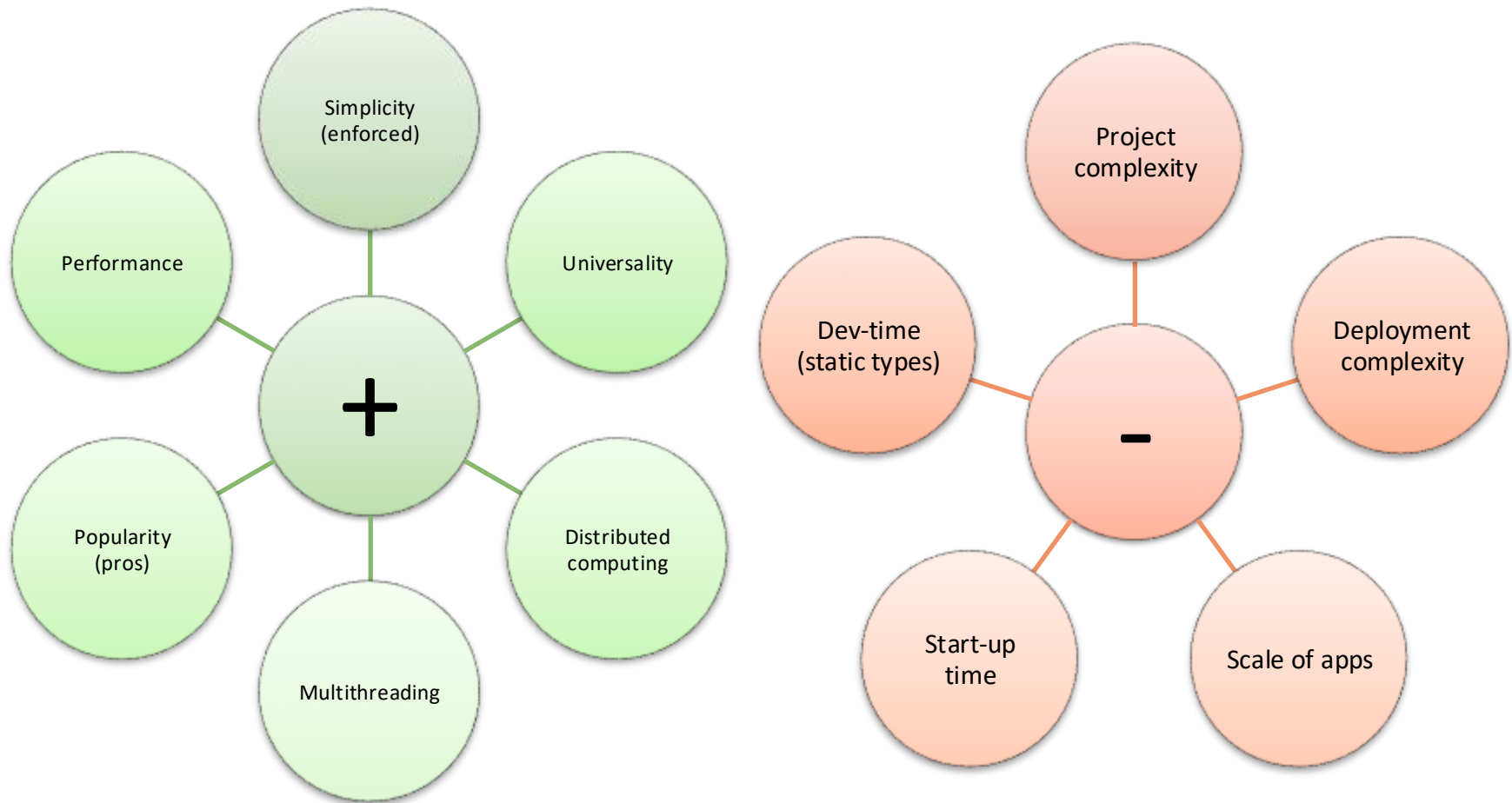
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

    @Bean
    public CommandLineRunner commandLineRunner(ApplicationContext ctx) {
        return args -> {
            System.out.println("Let's inspect the beans provided by Spring Boot:");

            String[] beanNames = ctx.getBeanDefinitionNames();
            Arrays.sort(beanNames);
            for (String beanName : beanNames) {
                System.out.println(beanName);
            }
        };
    }
} See: https://spring.io/guides/gs/spring-boot/
```

Java – Pros & Cons



JS on Server-side (Node.js) – Basics

- An open source, cross-platform runtime environment to create server-side applications in JavaScript
- Performance & scalability in web applications
- “*Plain old JavaScript*” on both sides
- Active community developing many additional tools&packages (at various stages of maturity)
- Designed for the event-driven systems

Node.js – Installation & preparation

- Install Node.js as an installer:
<https://nodejs.org/en/download/>
or via package manager:
<https://nodejs.org/en/download/package-manager/>
- Run Node.js via command line using node or use a configured IDE e.g.
<https://visualstudio.microsoft.com/vs/features/node-js/>

Node.js – Hello World

```
// Load HTTP module
const http = require("http");
```

```
const hostname = "127.0.0.1";
const port = 8000;
```

```
// Create HTTP server
const server = http.createServer(function(req, res) {
```

```
    // Set the response HTTP header with HTTP status and Content type
    res.writeHead(200, {'Content-Type': 'text/plain'});
```

```
    // Send the response body "Hello World"
    res.end('Hello World\n');
});
```

```
// Prints a log once the server starts listening
server.listen(port, hostname, function() {
    console.log(`Server running at http://${hostname}:${port}/`);
})
```

1. Create project folder
2. Save file
3. Run by: `node helloworld.js`

Node.js – Express

- A fast & minimalist web framework for Node.js
- Features sacrificed to gain performance (sic!)
- A foundation for many other frameworks

See: <https://expressjs.com/en/resources/frameworks.html>

- Used in MEAN (MongoDB, Express, Angular, Node.JS) and MERN (MongoDB, Express, React, Node.JS)
- Install by: `npm install express --save`

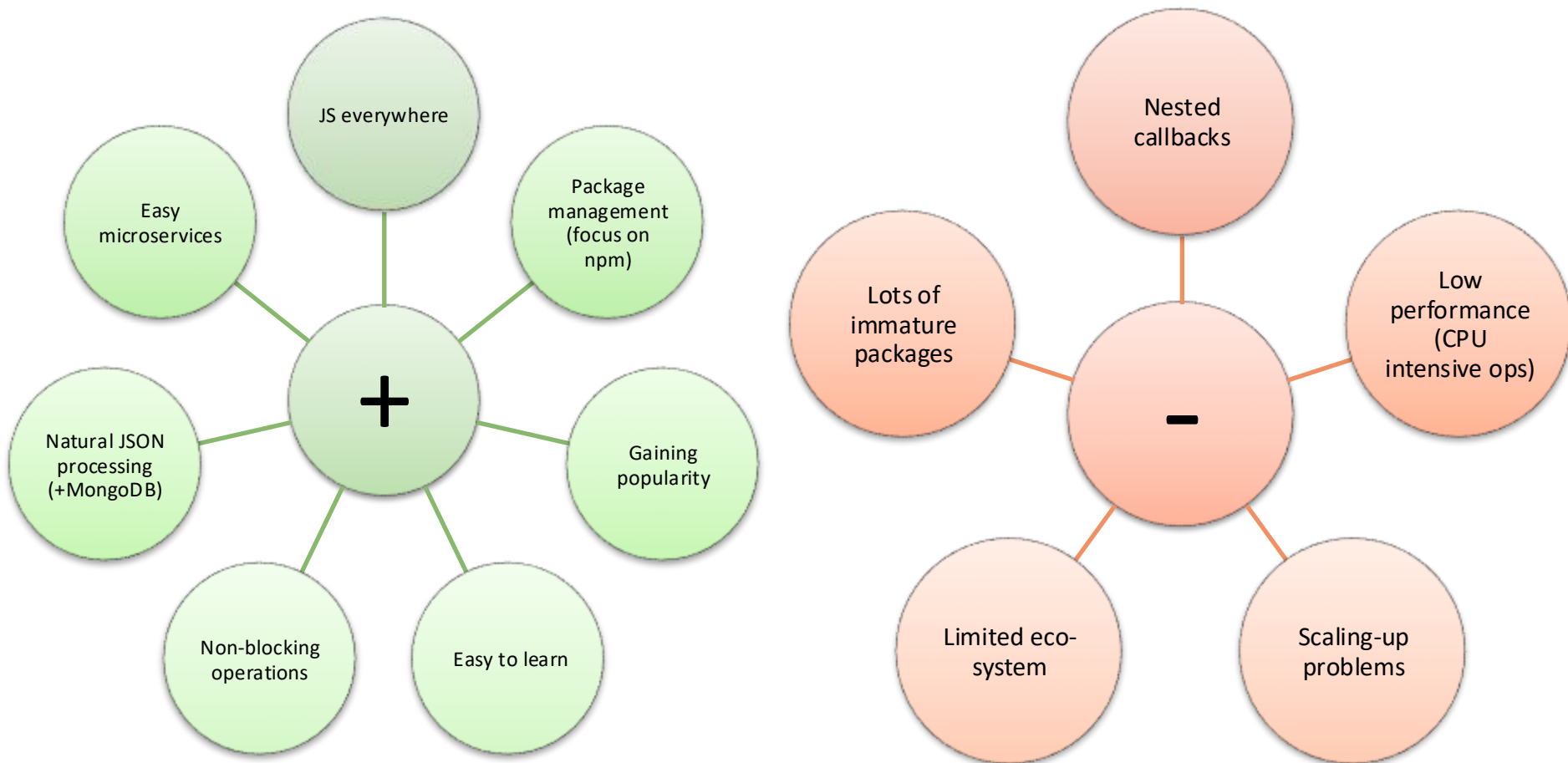
Express – Hello World

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', function(req, res) {
  res.send('Hello World!')
});

app.listen(port, function() {
  console.log(`Example app listening on port ${port}!`)
});
```

JS (Node.js) – Pros & Cons



Technologies – selection

PHP

- Easy start-up
- Supported by hosting packages
- HTML-mixture
- Popularity in small or mid-sized projects

Java

- Maturity
- Performance
- Capabilities
- Support – tools, soft & devs

Python

- Prototyping power
- Capabilities
- Popularity for various tasks (AI, data processing, API)
- Versatility

JavaScript (Node.js)

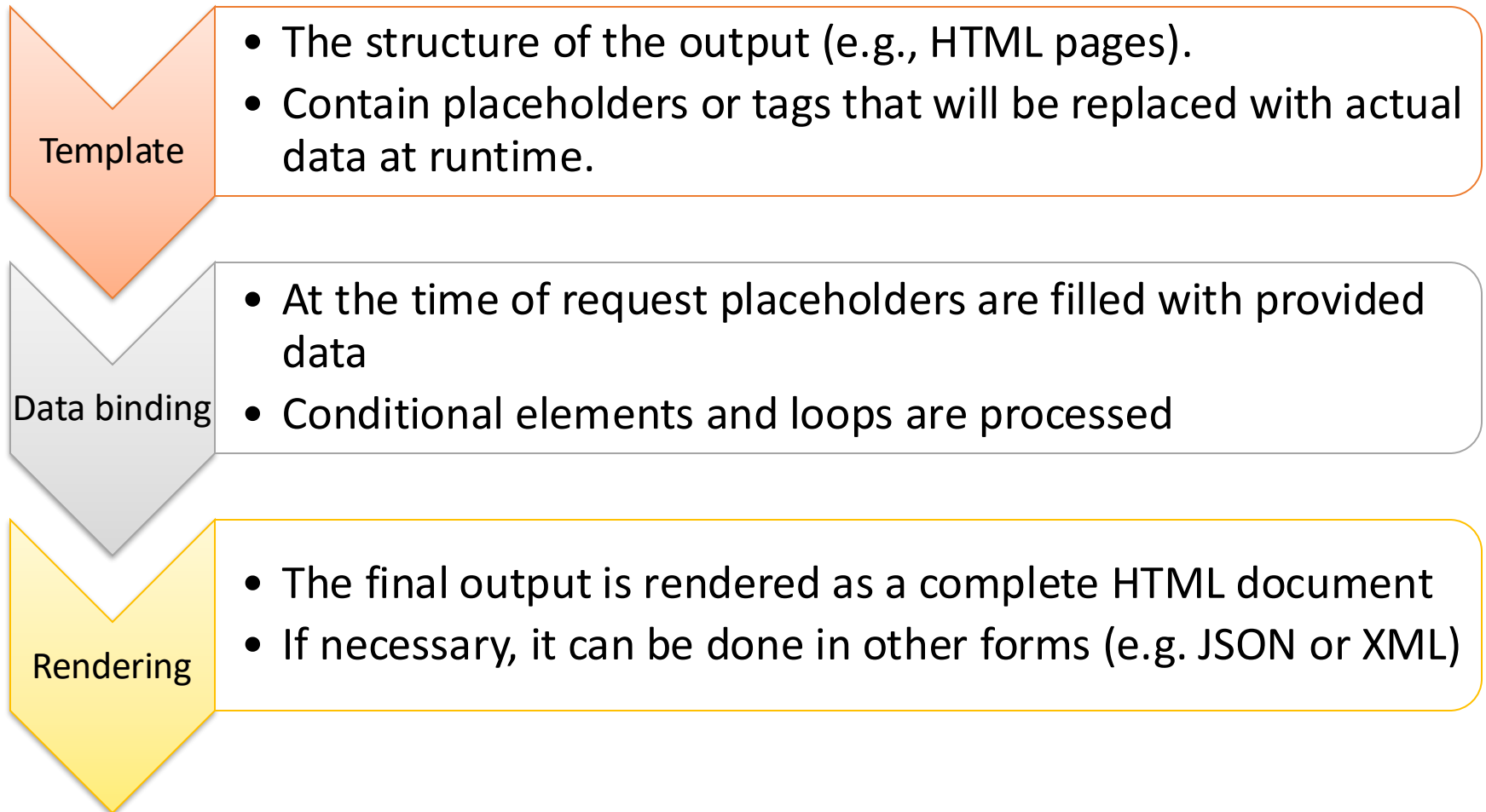
- JS on both ends (JS full-stack)
- Performance (massive number of simple requests)
- “Fashionable”
- Natural choice for Single-Page Apps

TEMPLATE ENGINES

Templating engines

- Software processing templates together with data model, and producing result document
- Separates system into layers
- Increases productivity by a usage of common elements combined with loops or logic
- A typical way to create web applications, especially when using frameworks

Templating – the pipeline



Mustache (multiple languages)

```
Hello {{name}}  
You have just won {{value}} dollars!  
{{#in_ca}}  
Well, {{taxed_value}} dollars, after taxes.  
{{/in_ca}}
```

```
{  
  "name": "Chris",  
  "value": 10000,  
  "taxed_value": 10000 - (10000 * 0.4),  
  "in_ca": true  
}
```

```
Hello Chris  
You have just won 10000 dollars!  
Well, 6000.0 dollars, after taxes.
```


Blade Engine - A part of Laravel (PHP)

```
<html>
  <head>
    <title>App Name - @yield('title')</title>
  </head>
  <body>
    @section('sidebar')
      This is the master sidebar.
    @show

    <div class="container">
      @yield('content')
    </div>
  </body>
</html>
```

See: <https://laravel.com/docs/8.x/blade>

Django templates (Python)

```
{% if latest_question_list %}
    <ul>
        {% for question in latest_question_list %}
            <li><a href="/polls/{{ question.id }}/">{{
question.question_text }}</a></li>
        {% endfor %}
    </ul>
{% else %}
    <p>No polls are available.</p>
{% endif %}
```

See:

<https://docs.djangoproject.com/en/4.0/topics/templates/>
<https://wiki.python.org/moin/Templating>

Pug (JavaScript) – Tags

Tags

By default, text at the start of a line (or after only white space) represents an HTML tag. Indented tags are nested, creating the tree structure of HTML.

```
ul
  li Item A
  li Item B
  li Item C
```

```
<ul>
  <li>Item A</li>
  <li>Item B</li>
  <li>Item C</li>
</ul>
```

Pug also knows which elements are self-closing:

```
img
```

```
<img />
```

Block Expansion

To save space, Pug provides an inline syntax for nested tags.

```
a: img
```

```
<a><img /></a>
```

Pug (JavaScript) – Text #1

Inline in a Tag

The easiest way to add plain text is *inline*. The first term on the line is the tag itself. Everything after the tag and one space will be the text contents of that tag. This is most useful when the plain text content is short (or if you don't mind lines running long).

```
p This is plain old <em>text</em> content
```

```
<p>This is plain old <em>text</em> conten
```

Literal HTML

Whole lines are also treated as plain text when they begin with a left angle bracket (`<`), which may occasionally be useful for writing literal HTML tags in places that could otherwise be inconvenient. For example, one use case is *conditional comments*. Since literal HTML tags do not get processed, they do not self-close, unlike Pug tags.

```
<html>

body
  p Indenting the body tag here would make
  p HTML itself isn't whitespace-sensitive

</html>
```

```
<html>

<body>
  <p>Indenting the body tag here would make
  <p>HTML itself isn't whitespace-sensitive
</body>

</html>
```

Pug (JavaScript) – Text #2

Piped Text

Another way to add plain text to templates is to prefix a line with a pipe character (`|`). This method is useful for mixing plain text with inline tags, as we discuss later, in the Whitespace Control section.

```
p
| The pipe always goes at the beginning
| not counting indentation.
```

```
<p>The pipe always goes at the beginning
not counting indentation.</p>
```

Block in a Tag

Often you might want large blocks of text within a tag. A good example is writing JavaScript and CSS code in the `script` and `style` tags. To do this, just add a `.` right after the tag name, or after the closing parenthesis, if the tag has `attributes`.

There should be no space between the tag and the dot. Plain text contents of the tag must be indented one level:

```
script.
  if (usingPug)
    console.log('you are awesome')
  else
    console.log('use pug')
```

```
<script>
  if (usingPug)
    console.log('you are awesome')
  else
    console.log('use pug')
</script>
```

Pug (JavaScript) – Attributes

Attributes

Tag attributes look similar to HTML (with optional commas), but their values are just regular JavaScript.

(NOTE: Examples on this page use the pipe character (`|`) for **whitespace control**.)

```
a(href='//google.com') Google
|
|
a(class='button' href='//google.com') Goo
|
|
a(class='button', href='//google.com') Go
```

```
<a href="//google.com">Google</a>
<a class="button" href="//google.com">Goo
<a class="button" href="//google.com">Goo
```

Normal JavaScript expressions work fine, too:

```
- var authenticated = true
body(class=authenticated ? 'authed' : 'an
```

```
<body class="authed"></body>
```

Pug (JavaScript) – Case

Case

The `case` statement is a shorthand for JavaScript's `switch` statement. It takes the following form:

```
- var friends = 10
case friends
  when 0
    p you have no friends
  when 1
    p you have a friend
  default
    p you have #{friends} friends
```

```
<p>you have 10 friends</p>
```

Pug (JavaScript) – Conditionals

Conditionals

Pug's first-class conditional syntax allows for optional parentheses.

If you're coming from Pug v1, you may now omit the leading `-`. Otherwise, it's identical (just regular JavaScript):

```
- var user = {description: 'foo bar baz'}
- var authorised = false
#user
  if user.description
    h2.green Description
    p.description= user.description
  else if authorised
    h2.blue Description
    p.description.
      User has no description,
      why not add one...
  else
    h2.red Description
    p.description User has no description
```

```
<div id="user">
  <h2 class="green">Description</h2>
  <p class="description">foo bar baz</p>
</div>
```

Pug also provides the conditional `unless`, which works like a negated `if`. The following are equivalent:

```
unless user.isAnonymous
  p You're logged in as #{user.name}
```

```
if !user.isAnonymous
  p You're logged in as #{user.name}
```


Pug (JavaScript) – Iterations #1

each

Pug's first-class iteration syntax makes it easier to iterate over arrays and objects in a template:

```
ul
  each val in [1, 2, 3, 4, 5]
    li= val
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
</ul>
```

You can also get the index as you iterate:

```
ul
  each val, index in ['zero', 'one', 'two']
    li= index + ': ' + val
```

```
<ul>
  <li>0: zero</li>
  <li>1: one</li>
  <li>2: two</li>
</ul>
```

Pug also lets you iterate over the keys in an object:

```
ul
  each val, key in {1: 'one', 2: 'two', 3: 'three'}
    li= key + ': ' + val
```

```
<ul>
  <li>1: one</li>
  <li>2: two</li>
  <li>3: three</li>
</ul>
```

Pug (JavaScript) – Iterations #2

while

You can also use `while` to create a loop:

```
- var n = 0;  
ul  
  while n < 4  
    li= n++
```

```
<ul>  
  <li>0</li>  
  <li>1</li>  
  <li>2</li>  
  <li>3</li>  
</ul>
```

Pug (JavaScript) – Interpolations #1

String Interpolation, Escaped

Consider the placement of the following template's locals: `title`, `author`, and `theGreat`.

```
- var title = "On Dogs: Man's Best Friend";  
- var author = "enlore";  
- var theGreat = "<span>escape!</span>";
```

```
h1= title  
p Written with love by #{author}  
p This will be safe: #{theGreat}
```

```
<h1>On Dogs: Man's Best Friend</h1>  
<p>Written with love by enlore</p>  
<p>This will be safe: &lt;span&gt;escape!</p>
```

`title` follows the basic pattern for evaluating a template local, but the code in between `#{` and `}` is evaluated, escaped, and the result buffered into the output of the template being rendered.

Pug (JavaScript) – Interpolations #2

String Interpolation, Unescaped

You don't *have* to play it safe. You can buffer unescaped values into your templates, too.

```
- var riskyBusiness = "<em>Some of the gi  
.quote  
p Joel: !{riskyBusiness}
```

```
<div class="quote">  
  <p>Joel: <em>Some of the girls are wear  
</div>
```

Caution

Keep in mind that buffering unescaped content into your templates can be mighty risky if that content comes fresh from your users. Never trust user input!

Pug (JavaScript) – Interpolations #3

Tag Interpolation

Interpolation works not only on JavaScript values, but on Pug as well. Just use the tag interpolation syntax, like so:

```
p.  
  This is a very long and boring paragra  
  Suddenly there is a #[strong strongly w  
  #[em ignored].  
p.  
  And here's an example of an interpolate  
  #[q(lang="es") iHola Mundo!]
```

```
<p>This is a very long and boring paragra  
  Suddenly there is a <strong>strongly wo  
  <em>ignored</em>.</p>  
<p>And here's an example of an interpolat  
  <q lang="es">iHola Mundo!</q></p>
```

You could accomplish the same thing by writing an HTML tag inline with your Pug...but then, what's the point of writing the Pug? Wrap an inline Pug tag declaration in `#[` and `]`, and it'll be evaluated and buffered into the content of its containing tag.

The main concerns relating to the templating

Performance

- Adds overhead due to the additional tasks to be done
- Pre-compilation of templates
- Potential caching of results (detection of changes)
- Inefficient data access (e.g. repeated DB queries)
- Additional layer increasing complexity and effecting manageability
- The more features it has, the less performant engine is

Security

- Potential attacks via:
 - scripts (Cross-site scripting),
 - injection (SQLs, templates),
 - deliberate errors.
- Cleansing (sanitisation) must be performed (additional task)
- Denial of Service due to the complexity of rendering
- Usually provided by mature template engines

WEB FRAMEWORKS

In short – Do not reinvent ...



Web frameworks – Main ideas #1

Abstraction & simplification

- Frameworks provide reusable components and libraries, reducing the need to write repetitive code.
- Abstract away low-level details, allowing developers to focus on application logic rather than underlying protocols and technologies.

MVC Architecture (and other patterns)

- Many implement the MVC design pattern, separating application logic (Model), user interface (View), and user input (Controller) for better organization and maintainability.

URL routing

- Provide mechanisms to define routes that map URLs to specific functions or controllers, simplifying the handling of incoming requests.

Web frameworks – Main ideas #2

Middleware Support

- Often support middleware, which allows developers to process requests and responses at various stages of the request lifecycle (e.g., authentication, logging).

Database Integration

- Often include ORM tools that facilitate database interactions, allowing developers to work with databases using high-level programming constructs instead of raw SQL.

Security Features

- Often come with built-in security features, such as protection against common vulnerabilities (e.g., XSS, CSRF, SQL injection) and user authentication mechanisms.

Template Engines

- Typically include templating engines that allow developers to generate dynamic HTML content by combining templates with data.

Web frameworks – Main ideas #3

Session and State Management

- Provide tools for managing user sessions and maintaining state across multiple requests, which is essential for user authentication and personalized experiences.

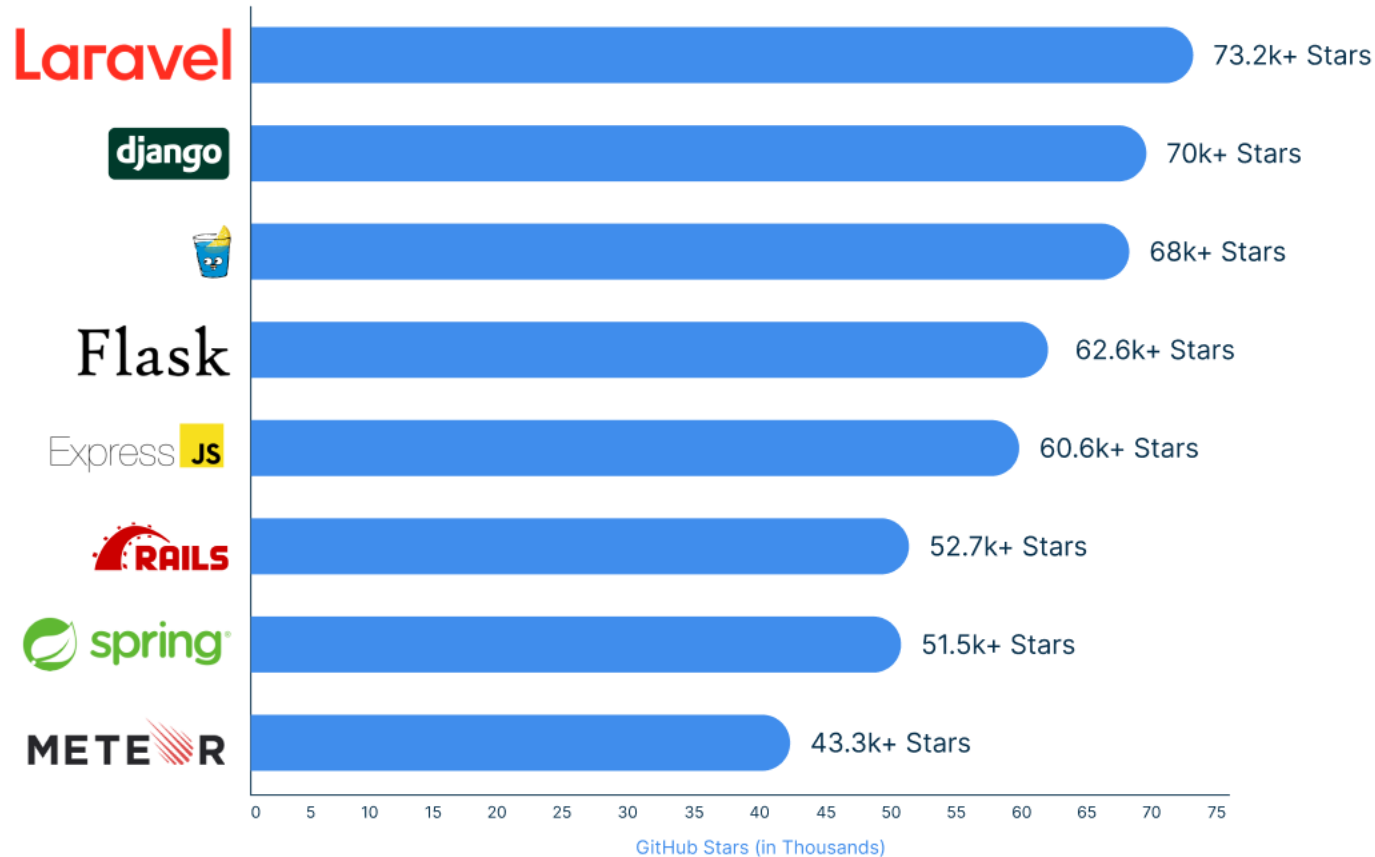
Testing and Debugging Tools

- Many include testing tools and libraries to facilitate unit testing, integration testing, and debugging, promoting better code quality.

Community and Ecosystem

- Popular ones have large communities that contribute to documentation, plugins, and third-party libraries, enhancing the framework's capabilities.
- Usually come with a rich ecosystem of tools, plugins, and extensions that can be easily integrated to add functionality.

Web frameworks – Popularity in 2024 (Mindinventory.com – based on GitHub stars)



Popular frameworks – Descriptions

LANGUAGE	FRAMEWORK	DESCRIPTION
PHP	Laravel	A popular PHP framework that provides an elegant syntax and a rich set of features for building web applications, including routing, authentication, and ORM.
Python	Django	A high-level Python web framework that encourages rapid development and clean, pragmatic design, featuring an integrated admin panel and ORM.
	Flask	A lightweight WSGI web application framework that is easy to get started with and highly extensible, ideal for small to medium-sized applications.
JavaScript	Express.js	A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications, focusing on simplicity and performance.
Ruby	Ruby on Rails	A full-stack web application framework that follows the convention over configuration principle, making it easy to develop database-backed web applications with built-in tools for testing and deployment.
Java	Spring	A comprehensive framework for building Java applications, providing a wide range of features for web development, including dependency injection, security, and data access.
Go	Gin Gonic	A web framework written in Go that is known for its speed and small memory footprint, suitable for building high-performance applications with a focus on simplicity and efficiency.

The comparison of performance

Languages - <https://benchmarksgame-team.pages.debian.net/benchmarksgame/index.html>

Frameworks - <https://www.techempower.com/benchmarks/>

Dynamic Documents and Front-end Web Development

? [Disable all](#)

C#

C++

cfml

Clojure

common lisp

Dart

Elixir

Erlang

f#

fortran

Haskell

Java

JavaScript

julia

Kotlin

ocaml

pascal

Perl

PHP

prolog

racket

Ruby

Rust

Scala

Swift

v

vala

vb

The Server-side and Development Environment

Dynamic Documents and Front-end Web Development

cfml	chi	cinatra	clevergo	codeigniter	coldbox	Collie	colossus	comet	compojure	cpoll-cppsp
crax	cuehttp	curacao	cutelyst	dia	ditsmod	django	donkey	dragon	dropwizard	duckphp
duct	easyrpc	echo	elli	elysia	embedio	emmett	es4x	eve	evio	express
facil.io	faf	falco	falcon	fastapi	fastendpoints	fastify	Fat Free	feathersjs	ffead-cpp	fiber
finagle	finatra	finch	fintrospect	flask	flight	fortran.io	frank	fuel	gearbox	gemin
genhttp	gin	gin-gorm	giraffe	gnet	go-std	goev	goframe	Goji	gotham	grails
granian	grape	greenlightning	grip	grizzly	h2o.cr	hapi	helidon	hertz	hertz-gorm	Hexagon
Hot	hserver	http.jl	http4k	http4s	hug	hummingbird	hummingbirdcor	hyper	hyperexpress	ihp
imi	inverno	isocket-nio	japronto	jaavalin	jawn	Jersey	jester	jewel	jooby	just
kami	kemal	kit	Klein	koa	kooby: jooby+ko	ktor	kumbiaphp	lapis	laravel	leaf
libh2o	libhttpserver	libreactor	lithium	lucky	Lumen	luminus	luxure	lwan	magician	mako
mark	martian	martini	may-minihttp	microdot	microhttp	micronaut	minijax	mixphp	mochiweb	mojolicious
morepath	mormot	nancy	nestjs	netcoreserver	nette	netty	ningle	ninglex	ninja	njs
nodejs	None	ntex	oatpp	officefloor	ohkami	one	onyx	openresty	openswoole	padrino
pavex	pedestal	pekko-http	pellet	phalcon	Phoenix	php	php-ngx	phpixie	pico.v	pippo
plack	play1	play2	plug	plumber	polkadot	postgrest	prologue	proteus	pyramid	quarkus
quart	rack	rack-sequel	racket	rails	rapidoid	rapidoid-http-fa	ratpack	raze	reactor	reactphp
redkale	reitit	restana	restify	Revenj	Revenj.JVM	ringo	roa	robyn	roda-sequel	ronykit
salvo	saphir	scalene	scorper	servicestack-v6	servicetalk	silicon	silverlining	simple-server	simps	sinatra
sinatra-sequel	slim	solon	spark	spider-gazelle	spiral	spliffy	spring	starlette	starlite	stdlib
suave	suil	swi-prolog	swoole	symfony	tapestry	tetsu	tio-mvc	toro	treefrog	trillium

The Server-side and Development Environment

Dynamic Documents and Front-end Web Development

actix-http	actix-server	actix-web	agoo	aioworkers.n	bandit	beetlex
bjoern	blaze	caddy	cowboy	elli	embedio	falcon
fastwsgi	ffead-cpp	genhttp	granian	grizzly	Gunicorn	h2o
http.jl	hyper	hypercorn	Hypnotoad	Jetty	kestrel	lwan
may-minihtt	Meinheld	microhttpd	mochiweb	Monkey	netcoreserve	netty
nginx	nginx-uWSG	None	ntex	ohkami	Passenger	picov
Puma	racket	redkale	Resin	smarthttp	socketify.py	socketify.py-
socketify.py-	socketify.py-	soup	swi-prolog	tetsu	tomcat	Tornado
Twisted Web	Undertow	Unicorn	uvicorn	uWSGI	vert.x	vertx
wai	waitress	Wildfly	woo	woof	xitca-server	zysocket-v

The Server-side and Development Environment

Frameworks – Top 20 & Selected ones

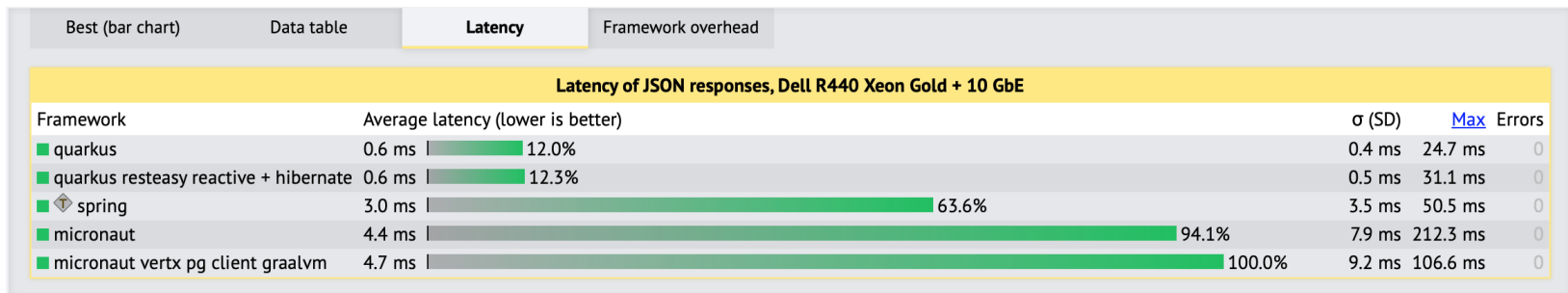
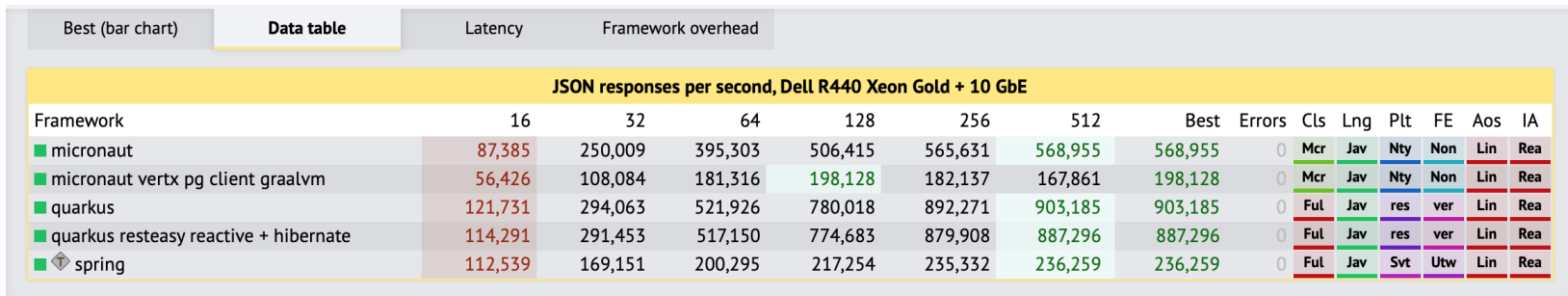
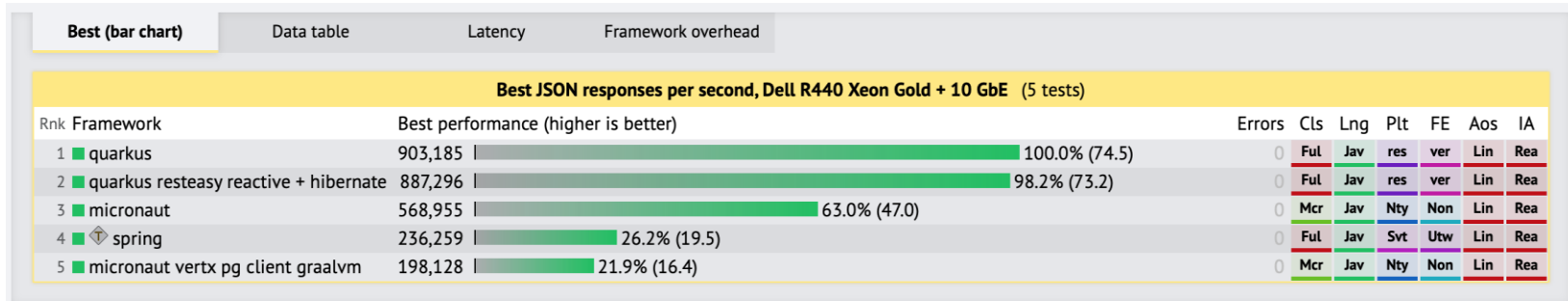
Composite Framework Scores

Each framework's peak performance in each test type (shown in the colored columns below) is multiplied by the weights shown above. The results are then summed to yield a weighted score. Only frameworks that implement all test types are included. 159 total frameworks ranked, 156 visible, 3 hidden by filters. See filter panel above.

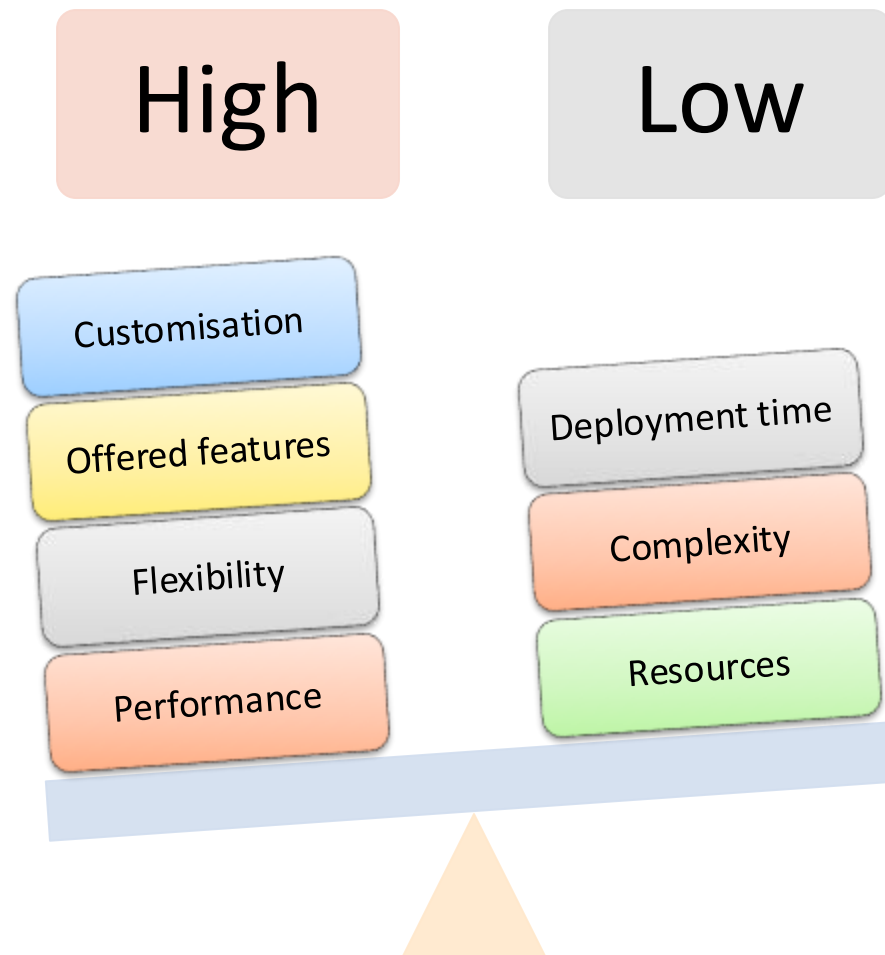
Rnk	Framework	JSON	1-query	20-query	Fortunes	Updates	Plaintext	Weighted score
1	 ntex	1,198,855	628,505	34,585	558,331	23,801	7,000,539	8,087 100.0%
2	 may-minihttp	1,205,461	617,258	32,361	585,122	22,909	6,909,552	8,059 99.7%
3	 xitca-web	1,171,415	613,534	32,369	578,573	23,007	6,612,565	7,950 98.3%
4	 just	1,123,167	612,670	32,447	422,871	23,009	6,961,689	7,324 90.6%
5	 redkale	1,210,086	457,935	32,272	413,537	22,900	6,981,831	7,096 87.8%
6	 libh2o	1,180,947	613,512	31,241	549,078	19,559	2,307,935	6,877 85.0%
7	 vert.x	1,194,427	581,542	32,505	453,406	17,122	5,624,745	6,847 84.7%
8	 officefloor	1,141,373	474,841	30,273	367,623	21,688	5,648,781	6,526 80.7%
9	 viz	800,219	539,816	32,630	432,605	22,881	4,085,883	6,441 79.6%
10	 axum	777,176	547,573	32,591	438,996	22,588	3,833,619	6,395 79.1%
11	 jooby	1,162,248	515,795	32,196	427,682	15,889	4,801,653	6,370 78.8%
12	 mormot	1,112,541	375,090	31,023	348,436	19,390	6,970,546	6,320 78.2%
13	 actix	1,194,185	429,376	22,538	405,144	15,658	6,970,300	6,288 77.8%
14	 wizzardo-http	1,162,363	552,613	28,942	306,558	16,454	6,995,129	6,266 77.5%
15	 asp.net core	1,042,029	392,709	25,329	363,344	18,197	7,014,298	6,143 76.0%
16	 vertx-web	1,121,889	566,804	30,669	388,512	16,598	3,774,206	6,107 75.5%
17	 salvo	788,392	541,576	32,505	413,812	22,760	2,042,846	6,011 74.3%
18	 lithium	1,210,365	353,405	18,020	351,475	14,204	6,993,976	5,760 71.2%
19	 inverno	1,042,404	467,759	32,441	309,684	15,864	4,043,496	5,566 68.8%
20	 ffead-cpp	1,119,233	352,023	23,235	329,657	12,708	6,085,971	5,440 67.3%

Rnk	Framework	JSON	1-query	20-query	Fortunes	Updates	Plaintext	Weighted score
38	 quarkus	903,185	318,897	17,610	214,275	6,697	2,861,479	3,637 45.0%
40	 micronaut	568,955	221,741	28,171	179,741	15,209	1,327,013	3,555 44.0%
86	 fastapi	366,204	72,724	11,373	46,896	7,610	448,130	1,524 18.8%
88	 spring	236,259	147,907	15,932	24,082	7,131	506,087	1,507 18.6%
104	 nodejs	320,168	95,776	6,860	70,833	3,048	454,082	1,206 14.9%
138	 laravel	77,648	37,275	4,656	22,501	1,666	81,052	462 5.7%
143	 django	177,099	19,032	1,623	14,707	871	300,170	413 5.1%

Frameworks – Single test for selected ones



Framework selection – Expectations



DEVELOPMENT & OPERATING ENVIRONMENT

IDE - Functions

- Integrated Development Environment facilitates efficient software development
- It could be an application providing features by itself or build over a set of tool providing these

Required (*de facto*):

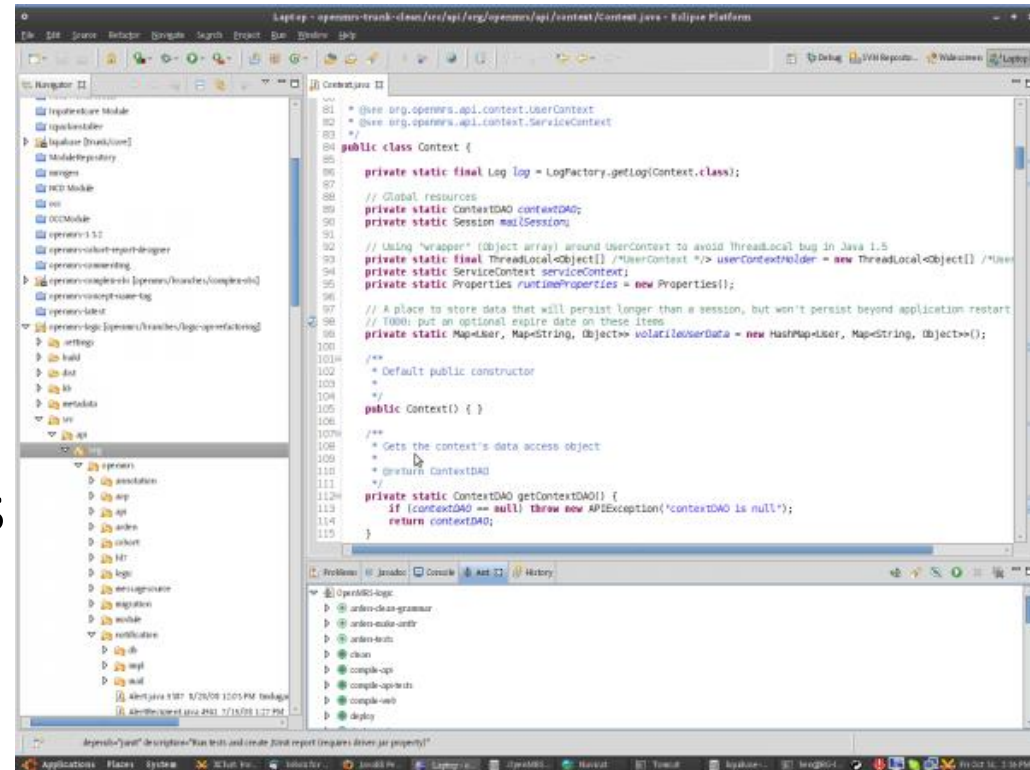
- Source code editing
- Building and deployment
- Testing and debugging
- File management

Expected:

- Version control
- Refactoring
- Issue tracking
- Syntax highlighting
- Code completion

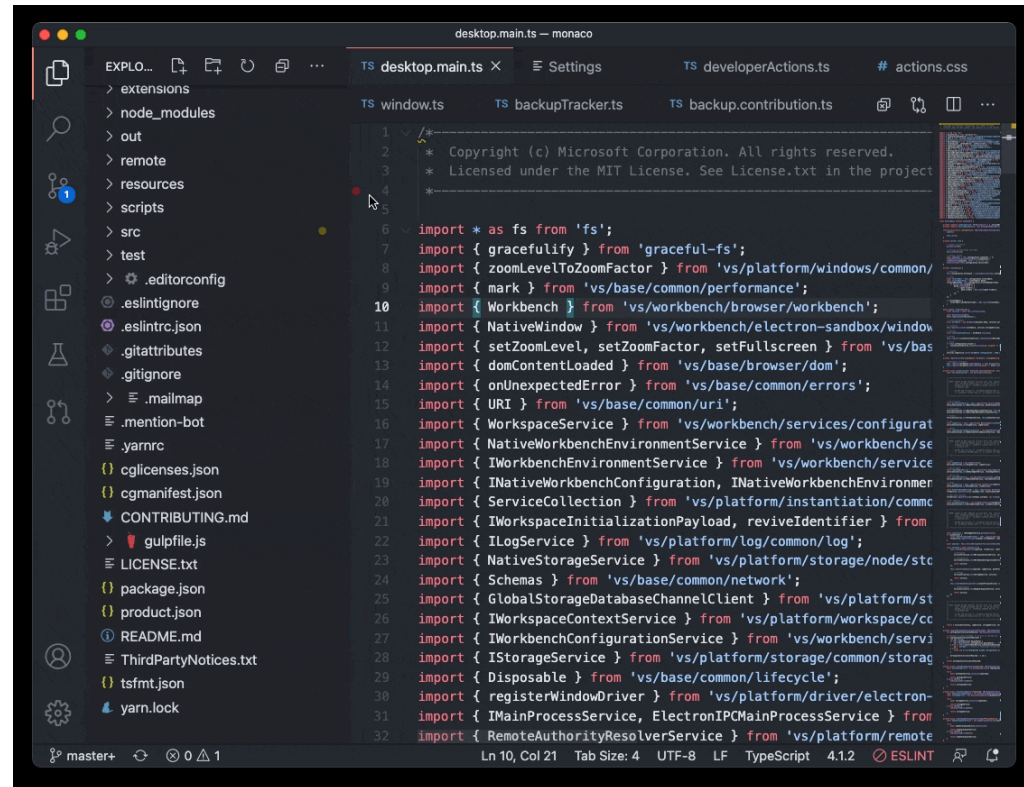
IDE - Eclipse

- OS led by Eclipse Foundation
- Founded in 2001
- The most versatile IDE
- Different supported languages, technologies and documents + tools
- Rich client platform to build other apps



IDE – Visual Studio Code

- Developed by Microsoft
- Supported multiple programming languages
- An Electron app + Node.js + Blink layout
- Less powerful, but lightweight comparing to Eclipse



Simpler development solutions

Minimal IDE

- Simplest usable solutions
- Low resource requirements
- No/minimal configuration
- ~~Global minimum = command prompt~~

Brackets / Phoenix Code

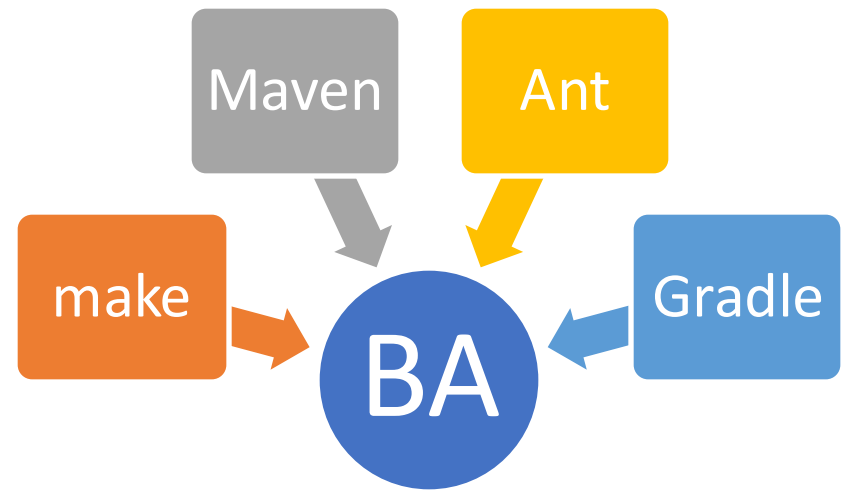
- Web-oriented editor (front-end)
- Live preview functionality
- Extensions to work with other languages

Geany / Notepad++

- ... any other programming editor
- Light text editors
- Supporting basic editing of source code
- Compilation & etc. done externally

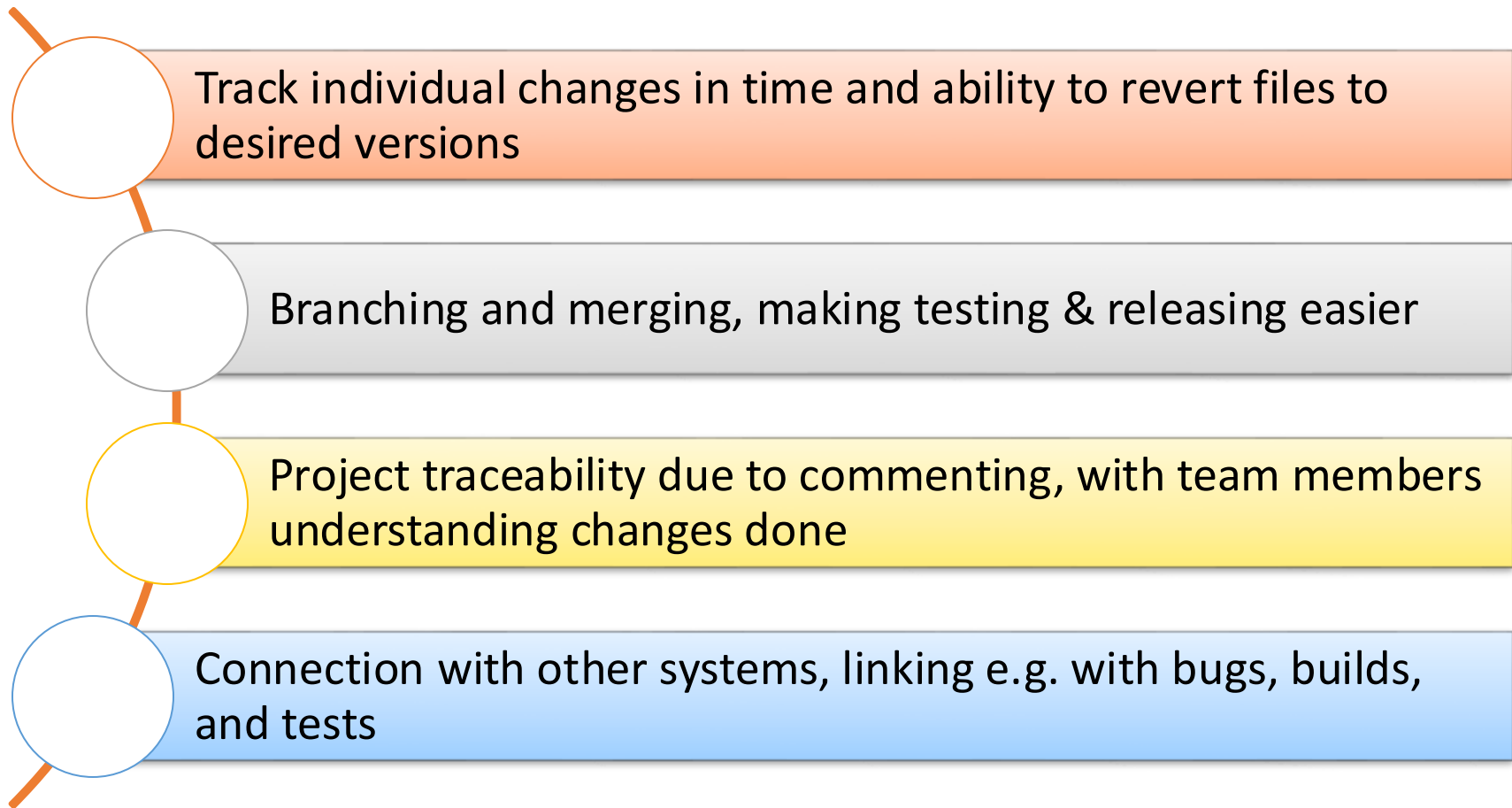
Build automation & dependency management

- Controls process of project building
- Defines dependencies
- Controls versions
- Basic for continuous development
- History of builds
- Release profiles
- Deployment included



See: <https://search.maven.org>

Version control



Version control – Popular solutions

Git (a default choice)

- Current number 1 among VCSes (>90%)
- Distributed VCS – local and remote repositories
- Lightweight branches – management of them doesn't affect commits
- Less controllable permissions
- More complex command line syntax
- GUI clients & extensions for IDEs

Other choices

- Subversion (SVN)
- Mercurial (Git's strong competitor at early stage)
- Legacy projects or used by large companies

Packages & Containers & Virtual Machines

Package	Container	Virtual Machine
• Files with metadata used to install & configure app in OS • Might depend on other packages • Managed by a tool controlling installed packages and dependencies • Dnf, yum or snap	• Isolated user space (OS-virtualisation) • Programmes operate in OS, with access to a part of resources (other apps, libraries) • Docker, containerd, CRI-O or Kubernetes	• Guest OS with apps run in an isolated space • Controllable available hardware resources • VirtualBox, KVM, QEMU or Xen

P & C & VM – Example (Packages)

Exact hits

Package apache2

- **focal (20.04LTS)** (web): Apache HTTP Server
2.4.41-4ubuntu3.21 [**security**]: amd64 i386
2.4.41-4ubuntu3 [**ports**]: arm64 armhf ppc64el riscv64 s390x
- **focal-updates** (web): Apache HTTP Server
2.4.41-4ubuntu3.21: amd64 arm64 armhf i386 ppc64el riscv64 s390x
- **jammy (22.04LTS)** (web): Apache HTTP Server
2.4.52-1ubuntu4.12 [**security**]: amd64 i386
2.4.52-1ubuntu4 [**ports**]: arm64 armhf ppc64el riscv64 s390x
- **jammy-updates** (web): Apache HTTP Server
2.4.52-1ubuntu4.12: amd64 arm64 armhf i386 ppc64el riscv64 s390x
- **noble (24.04LTS)** (web): Apache HTTP Server
2.4.58-1ubuntu8.4 [**security**]: amd64 i386
2.4.58-1ubuntu8 [**ports**]: arm64 armhf ppc64el riscv64 s390x
- **noble-updates** (web): Apache HTTP Server
2.4.58-1ubuntu8.5: amd64 arm64 armhf i386 ppc64el riscv64 s390x
- **oracular (24.10)** (web): Apache HTTP Server
2.4.62-1ubuntu1: amd64 arm64 armhf i386 ppc64el riscv64 s390x
- **plucky** (web): Apache HTTP Server
2.4.62-3ubuntu1: amd64 arm64 armhf i386 ppc64el riscv64 s390x

Package mariadb-server

- **focal (20.04LTS)** (database): MariaDB database server (metapackage depending on the latest version) [**universe**]
1:10.3.39-0ubuntu0.20.04.2 [**security**]: all
- **focal-updates** (database): MariaDB database server (metapackage depending on the latest version) [**universe**]
1:10.3.39-0ubuntu0.20.04.2: all
- **jammy (22.04LTS)** (database): MariaDB database server (metapackage depending on the latest version) [**universe**]
1:10.6.18-0ubuntu0.22.04.1 [**security**]: all
- **jammy-updates** (database): MariaDB database server (metapackage depending on the latest version) [**universe**]
1:10.6.18-0ubuntu0.22.04.1: all
- **noble (24.04LTS)** (database): MariaDB database server binaries [**universe**]
1:10.11.8-0ubuntu0.24.04.1 [**security**]: amd64
1:10.11.7-2ubuntu2 [**ports**]: arm64 armhf ppc64el riscv64 s390x
- **noble-updates** (database): MariaDB database server binaries [**universe**]
1:10.11.8-0ubuntu0.24.04.1: amd64 arm64 armhf ppc64el riscv64 s390x
- **oracular (24.10)** (database): MariaDB database server binaries [**universe**]
1:11.4.3-1: amd64 arm64 armhf ppc64el riscv64 s390x
- **plucky** (database): MariaDB database server binaries [**universe**]
1:11.4.3-1: amd64 arm64 armhf ppc64el riscv64 s390x

P & C & VM – Example (Containers)



httpd Docker Official Image · 1B+ · 4.8K
The Apache HTTP Server Project

WEB SERVERS

Overview Tags

Quick reference

- Maintained by:
[the Docker Community](#)
- Where to get help:
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

- [2.4.62](#) · [2.4](#) · [2](#) · [latest](#) · [2.4.62-bookworm](#) · [2.4-bookworm](#) · [2-bookworm](#) · [bookworm](#)
- [2.4.62-alpine](#) · [2.4-alpine](#) · [2-alpine](#) · [alpine](#) · [2.4.62-alpine3.21](#) · [2.4-alpine3.21](#) · [2-alpine3.21](#) · [alpine3.21](#)

Quick reference (cont.)

- Where to file issues:
<https://github.com/docker-library/httpd/issues>
- Supported architectures: ([more info](#))
[amd64](#) · [arm32v5](#) · [arm32v6](#) · [arm32v7](#) · [arm64v8](#) · [i386](#) · [mips64le](#) · [ppc64le](#) · [riscv64](#) · [s390x](#)



mariadb Docker Official Image · 1B+ · 5.9K
MariaDB Server is a high performing open source relational database, forked from MySQL.

DATABASES & STORAGE

Overview Tags

Quick reference

- Maintained by:
[MariaDB Foundation](#), [MariaDB plc](#), with contributions from our [community](#)
- Where to get help:
[Database Administrators \(Stack Exchange\)](#), [MariaDB Knowledge Base](#) ([Ask a Question here](#))

Also see the ["Getting Help with MariaDB" article on the MariaDB Knowledge Base](#).

Supported tags and respective Dockerfile links

- [11.7.1-ubi9-rc](#) · [11.7-ubi9-rc](#) · [11.7.1-ubi-rc](#) · [11.7-ubi-rc](#)
- [11.7.1-noble-rc](#) · [11.7-noble-rc](#) · [11.7.1-rc](#) · [11.7-rc](#)
- [11.6.2-ubi9](#) · [11.6-ubi9](#) · [11-ubi9](#) · [11.6.2-ubi](#) · [11.6-ubi](#) · [11-ubi](#)
- [11.6.2-noble](#) · [11.6-noble](#) · [11-noble](#) · [noble](#) · [11.6.2](#) · [11.6](#) · [11](#) · [latest](#)
- [11.4.4-ubi9](#) · [11.4-ubi9](#) · [lts-ubi9](#) · [11.4.4-ubi](#) · [11.4-ubi](#) · [lts-ubi](#)
- [11.4.4-noble](#) · [11.4-noble](#) · [lts-noble](#) · [11.4.4](#) · [11.4](#) · [lts](#)
- [11.2.6-jammy](#) · [11.2-jammy](#) · [11.2.6](#) · [11.2](#)
- [10.11.10-ubi9](#) · [10.11-ubi9](#) · [10-ubi9](#) · [10.11.10-ubi](#) · [10.11-ubi](#) · [10-ubi](#)
- [10.11.10-jammy](#) · [10.11-jammy](#) · [10-jammy](#) · [10.11.10](#) · [10.11](#) · [10](#)
- [10.6.20-ubi9](#) · [10.6-ubi9](#) · [10.6.20-ubi](#) · [10.6-ubi](#)
- [10.6.20-focal](#) · [10.6-focal](#) · [10.6.20](#) · [10.6](#)
- [10.5.27-focal](#) · [10.5-focal](#) · [10.5.27](#) · [10.5](#)

P & C & VM – Example (VM)



Bitnami package for LAMP

php.net

DEPLOYMENT OFFERING



On the cloud

Single-Tier



On my computer

Virtual Machines

ABOUT BITNAMI PACKAGE FOR LAMP

Trademarks: This software listing is packaged by Bitnami. The respective trademarks mentioned in the offering are owned by the respective companies, and use of them does not imply any affiliation or endorsement.

LAMP packaged by Bitnami provides a complete, fully-integrated and ready to run LAMP development environment. In addition to PHP, MariaDB and Apache, it includes phpMyAdmin, openssl, ModSecurity, PageSpeed, Varnish, SQLite, ImageMagick, xDebug, Xcache, OpenLDAP, Memcache, OAuth, PEAR, PECL, APC, GD, cURL and Composer.

Download [virtual machines](#) or run your own lamp server [in the cloud](#).

Dynamic Documents and Front-end Web Development

P & C & VM – Comparison

ASPECT	SOFTWARE PACKAGES	CONTAINERS	VIRTUAL MACHINES (VMS)
Isolation	Limited isolation; may conflict with other packages.	Strong isolation; applications run in separate environments sharing host OS kernel.	Strong isolation; each VM has its own OS kernel.
Resources	😊	😐	😞
Performance	😊	😐	😞
Portability	😐	😊	😊
Deployment	Quick installation; may require configuration.	Fast to start and stop; quick deployment and scaling.	Slower to boot up and shut down compared to containers.
Management	Simple management; package managers handle dependencies.	Requires orchestration tools (e.g., Kubernetes) for complex deployments.	More complex management; requires hypervisor management and VM orchestration.
Security	Security depends on the underlying OS and package management.	Good security through isolation, but vulnerabilities in the host OS can affect containers.	Strong security isolation, but VMs can be vulnerable to hypervisor attacks.
Updates and Maintenance	Updates can be complex due to dependency management.	Easy to update and roll back; images can be versioned.	Updates can be cumbersome; each VM may need to be updated individually.

The Server-side and Development Environment

Failover & Load balancing

- Failover – switching to a redundant computer upon a failure
- Failback – process reverse to failover
- Main issues:
 - Data consistency
 - Cost of system redundancy
 - Failover detection
 - Scope of disaster
- Load balancing is a distribution of task over a set of nodes increasing the processing efficiency
- Possible solutions:
 - DNS Round-robin
 - DNS Least-connections
 - Geo-distributed redundant DNS delegation
 - Random

Summary

- Select server-side technology evaluating project & team (personal) characteristics
- There is no generally-superior technology, programming language or tool
- Prepare an efficient development & operating environment
- Planning for potential issues remember about application monitoring, protective measures & failure scenarios

TILL NEXT LECTURE